Spring Cloud Netflix Hystrix

主要内容

Hystrix 简介
服务降级
服务熔断
请求缓存
Feign 的雪崩处理
可视化的数据监控 Hystrix-dashboard

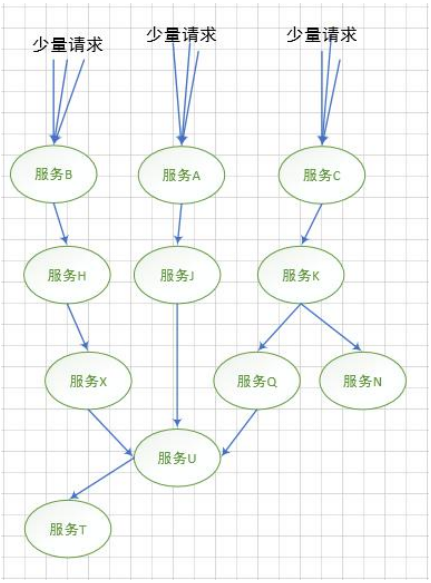
学习目标

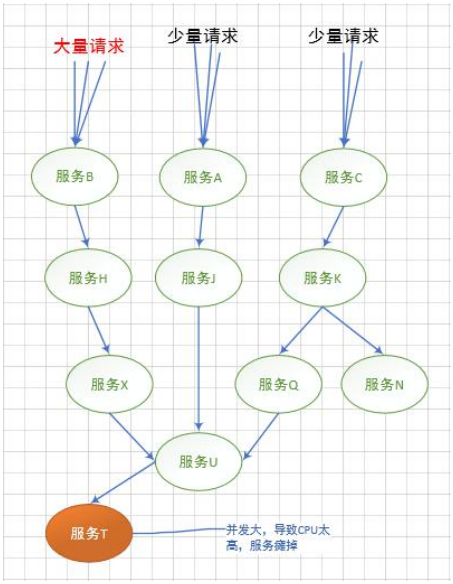
知识点	要求
Hystrix 简介	了解
服务降级	掌握
服务熔断	掌握
请求缓存	掌握
Feign 的雪崩处理	掌握
可视化的数据监控 Hystrix-dashboard	掌握

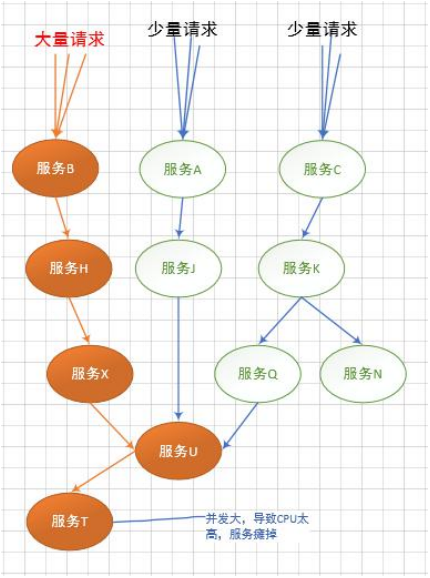
Spring Cloud Netflix Hystrix

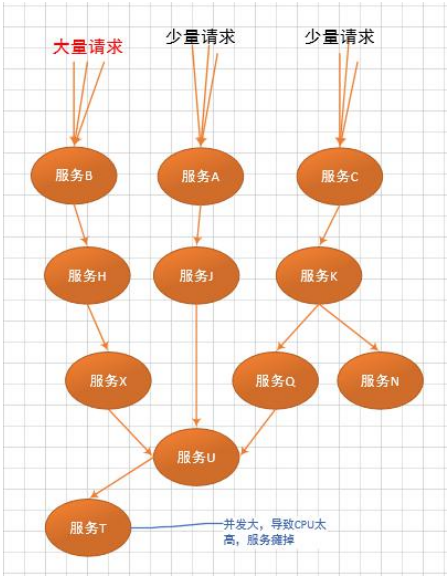
一、 Hystrix 简介

1 什么是灾难性雪崩效应









造成灾难性雪崩效应的原因，可以简单归结为下述三种：

服务提供者不可用。如：硬件故障、程序 BUG、缓存击穿、并发请求量过大等。

重试加大流量。如：用户重试、代码重试逻辑等。

服务调用者不可用。如：同步请求阻塞造成的资源耗尽等。

雪崩效应最终的结果就是：服务链条中的某一个服务不可用，导致一系列的服务不可用，最终造成服务逻辑崩溃。这种问题造成的后果，往往是无法预料的。

解决灾难性雪崩效应的方式通常有：降级、熔断和请求缓存。

2 什么是 Hystrix

Hystrix [hɪst'rɪks]，中文含义是豪猪，因其背上长满棘刺，从而拥有了自我保护的能力。本文所说的 Hystrix 是 Netflix 开源的一款容错框架，同样具有自我保护能力。为了实现容错和自我保护，下面我们看看 Hystrix 如何设计和实现的。

Hystrix 设计目标：

1、 对来自依赖的延迟和故障进行防护和控制——这些依赖通常都是通过网络访问的

2、 阻止故障的连锁反应

3、 快速失败并迅速恢复

4、 回退并优雅降级

5、 提供近实时的监控与告警

Hystrix 遵循的设计原则：

1、 防止任何单独的依赖耗尽资源（线程）

2、 过载立即切断并快速失败，防止排队

3、 尽可能提供回退以保护用户免受故障

4、 使用隔离技术（例如隔板，泳道和断路器模式）来限制任何一个依赖的影响

5、 通过近实时的指标，监控和告警，确保故障被及时发现

6、 通过动态修改配置属性，确保故障及时恢复

7、 防止整个依赖客户端执行失败，而不仅仅是网络通信

Hystrix 如何实现这些设计目标？

1、 使用命令模式将所有对外部服务（或依赖关系）的调用包装在 HystrixCommand 或 HystrixObservableCommand 对象中，并将该对象放在单独的线程中执行；

2、 每个依赖都维护着一个线程池（或信号量），线程池被耗尽则拒绝请求（而不是让请求排队）。

3、 记录请求成功，失败，超时和线程拒绝。

4、 服务错误百分比超过了阈值，熔断器开关自动打开，一段时间内停止对该服务的所有请求。

5、 请求失败，被拒绝，超时或熔断时执行降级逻辑。

6、 近实时地监控指标和配置的修改。

在 Spring cloud 中处理服务雪崩效应，都需要依赖 hystrix 组件。在 Application Client 应用的 pom 文件中都需要引入下述依赖：

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

二、 服务降级

降级是指，当请求超时、资源不足等情况发生时进行服务降级处理，不调用真实服务逻辑，而是使用快速失败（fallback）方式直接返回一个托底数据，保证服务链条的完整，避免服务雪崩。

解决服务雪崩效应，都是避免 application client 请求 application service 时，出现服务调用错误或网络问题。处理手法都是在 application client 中实现。我们需要在 application client 相关工程中导入 hystrix 依赖信息。并在对应的启动类上增加新的注解@EnableCircuitBreaker，这个注解是用于开启 hystrix 熔断器的，简言之，就是让代码中的 hystrix 相关注解生效。

具体实现过程如下：

1 修改 application service 代码

修改 application service 工程中的代码，模拟超时错误。此模拟中，让服务端代码返回之前休眠 2000 毫秒，替代具体的复杂服务逻辑。

```
package com.bjsxt.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ServiceController {
    @GetMapping
    public Object first(){
        try {
            // 用于模拟远程服务调用延时。
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "测试 Spring Cloud Netflix Ribbon 开发服务提供者";
    }
}
```

2 application client POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>
```

```
    <artifactId>applicationclient</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
        </dependency>
    </dependencies>

</project>
```

3 application client 容错处理代码

```
package com.bjsxt.service.impl;

import com.bjsxt.service.ClientService;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class ClientServiceImpl implements ClientService {
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * @HystrixCommand - 开启 Hystrix 容错处理的注解。代表当前方法如果出现服务
调用问题，使用 Hystrix 容错处理逻辑来处理
```

```
     *  属性 fallbackMethod - 如果当前方法调用服务，远程服务出现问题的时候，调用本地的哪个方法得到托底数据。
     */
    @HystrixCommand(fallbackMethod = "downgradeFallback")
    @Override
    public String first() {
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/");

        System.out.println("本次访问的 service 是:  " + sb.toString());

        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.GET, null, type);
        String result = response.getBody();

        return result;
    }

    // 本地容错方法，只有远程服务调用过程出现问题的时候，才会调用此方法，获取托底数据。保证服务完整性。
    private String downgradeFallback(){
        return "服务降级方法返回托底数据";
    }
}
```

4 application client 配置文件 application.yml

```yaml
server:
  port: 8082

spring:
  application:
    name: application-client
```

```yaml
eureka:
  client:
    service-url:
      defaultZone:
        - http://localhost:8761/eureka/

hystrix:
  command:
    default:
      execution:
        timeout:
          # 如果 enabled 设置为 false，则请求超时交给 ribbon 控制,为 true,则超时作为容错根据
          enabled: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000 # 超时时间，默认 1000ms
```

5 application client 启动类

```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;

/**
 * @EnableCircuitBreaker - 开启 Hystrix 容错处理能力。
 *  如果不使用此注解，服务代码中的@HystrixCommand 注解无效。
 */
@SpringBootApplication
@EnableCircuitBreaker
public class ClientApp {
    public static void main(String[] args) {
        SpringApplication.run(ClientApp.class, args);
    }
}
```

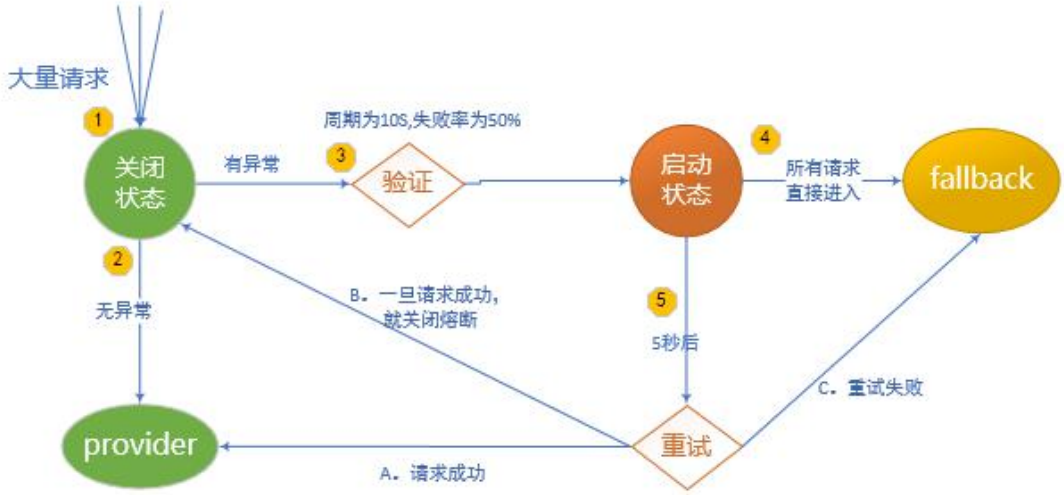
三、 服务熔断

当一定时间内，异常请求比例（请求超时、网络故障、服务异常等）达到阀值时，启

动熔断器，熔断器一旦启动，则会停止调用具体服务逻辑，通过 fallback 快速返回托底数据，保证服务链的完整。

熔断有自动恢复机制，如：当熔断器启动后，每隔 5 秒，尝试将新的请求发送给服务提供者，如果服务可正常执行并返回结果，则关闭熔断器，服务恢复。如果仍旧调用失败，则继续返回托底数据，熔断器持续开启状态。

熔断机制相当于电路的跳闸功能。
例如：我们可以配置熔断策略为当请求错误比例在10s内>50%时，该服务将进入熔断状态，后续请求都会进入fallback。



具体实现过程如下：

1 application client POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationclient</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
```

```
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
        </dependency>
    </dependencies>

</project>
```

2 application client 容错处理代码

```
package com.bjsxt.service.impl;

import com.bjsxt.service.ClientService;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixProperty;
import com.netflix.hystrix.contrib.javanica.conf.HystrixPropertiesManager;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class ClientServiceImpl implements ClientService {
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * @HystrixCommand - 开启 Hystrix 容错处理的注解。代表当前方法如果出现服务调用问题，使用 Hystrix 容错处理逻辑来处理
     *  属性 fallbackMethod - 如果当前方法调用服务，远程服务出现问题的时候，调用本地的哪个方法得到托底数据。
     */
    @HystrixCommand(fallbackMethod = "downgradeFallback",
            commandProperties = {
```

```
            // 默认 20 个;10s 内请求数大于 20 个时就启动熔断器，当请求符合熔断条件时将触发 fallback 逻辑
            @HystrixProperty(name = HystrixPropertiesManager.CIRCUIT_BREAKER_REQUEST_VOLUME_THRESHOLD, value = "10"),
            // 请求错误率大于 50%时就熔断，然后 for 循环发起请求，当请求符合熔断条件时将触发
            @HystrixProperty(name = HystrixPropertiesManager.CIRCUIT_BREAKER_ERROR_THRESHOLD_PERCENTAGE, value = "50"),
            // 默认 5 秒;熔断多少秒后去尝试请求
            @HystrixProperty(name = HystrixPropertiesManager.CIRCUIT_BREAKER_SLEEP_WINDOW_IN_MILLISECONDS, value = "5000")
            }
    )
    @Override
    public String first() {
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/");

        System.out.println("本次访问的 service 是： " + sb.toString());

        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.GET, null, type);
        String result = response.getBody();

        return result;
    }

    // 本地容错方法，只有远程服务调用过程出现问题的时候，才会调用此方法，获取托底数据。保证服务完整性。
    private String downgradeFallback(){
        return "服务降级方法返回托底数据";
    }
```

```
}
```

3 application client 配置文件 application.yml

```yaml
server:
  port: 8082

spring:
  application:
    name: application-client

eureka:
  client:
    service-url:
      defaultZone:
        - http://localhost:8761/eureka/

hystrix:
  command:
    default:
      execution:
        timeout:
          # 如果 enabled 设置为 false，则请求超时交给 ribbon 控制,为 true,则超时作为容错根据
          enabled: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000 # 超时时间，默认 1000ms
```

4 application client 启动类

```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;

/**
 * @EnableCircuitBreaker - 开启 Hystrix 容错处理能力。
 *  如果不使用此注解，服务代码中的@HystrixCommand 注解无效。
 */
@SpringBootApplication
```

```
@EnableCircuitBreaker
public class ClientApp {
    public static void main(String[] args) {
        SpringApplication.run(ClientApp.class, args);
    }
}
```

四、 请求缓存

请求缓存：通常意义上说，就是将同样的 GET 请求结果缓存起来，使用缓存机制（如 redis、mongodb）提升请求响应效率。

使用请求缓存时，需要注意非幂等性操作对缓存数据的影响。

请求缓存是依托某一缓存服务来实现的。在案例中使用 redis 作为缓存服务器，那么可以使用 spring-data-redis 来实现 redis 的访问操作。

1 修改 application service 代码

```
package com.bjsxt.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ServiceController {

    @PostMapping("/testPost")
    public Object testPost(){
        System.out.println("testPost method run");
        return "写操作返回";
    }

    @GetMapping("/testGet")
    public Object testGet(){
        System.out.println("testGet method run");
        return "读操作返回";
    }

    @GetMapping
    public Object first(){
        System.out.println("run");
        try {
            // 用于模拟远程服务调用延时。
```

```
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "测试 Spring Cloud Netflix Ribbon 开发服务提供者";
    }
}
```

2 application client POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationclient</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
        </dependency>
    </dependencies>
```

```
</project>
```

3 application client 容错处理代码

```
package com.bjsxt.service.impl;

import com.bjsxt.service.ClientService;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixProperty;
import com.netflix.hystrix.contrib.javanica.conf.HystrixPropertiesManager;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cache.annotation.CacheConfig;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

/**
 * 在类上，增加@CacheConfig 注解，用来描述当前类型可能使用 cache 缓存。
 * 如果使用缓存，则缓存数据的 key 的前缀是 cacheNames。
 * cacheNames 是用来定义一个缓存集的前缀命名的，相当于分组。
 */
@CacheConfig(cacheNames = "test.hystrix.caches")
@Service
public class ClientServiceImpl implements ClientService {
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * @HystrixCommand - 开启 Hystrix 容错处理的注解。代表当前方法如果出现服
务调用问题，使用 Hystrix 容错处理逻辑来处理
     *  属性 fallbackMethod - 如果当前方法调用服务，远程服务出现问题的时候，
调用本地的哪个方法得到托底数据。
     */
    @HystrixCommand(fallbackMethod = "downgradeFallback",
            commandProperties = {
            // 默认 20 个;10s 内请求数大于 20 个时就启动熔断器，当请求符合熔断
条件时将触发 fallback 逻辑
```

```
            @HystrixProperty(name =
HystrixPropertiesManager.CIRCUIT_BREAKER_REQUEST_VOLUME_THRESHOLD, value =
"10"),
            // 请求错误率大于 50%时就熔断，然后 for 循环发起请求，当请求符合熔
断条件时将触发
            @HystrixProperty(name =
HystrixPropertiesManager.CIRCUIT_BREAKER_ERROR_THRESHOLD_PERCENTAGE, value =
"50"),
            // 默认 5 秒;熔断多少秒后去尝试请求
            @HystrixProperty(name =
HystrixPropertiesManager.CIRCUIT_BREAKER_SLEEP_WINDOW_IN_MILLISECONDS,
value = "5000")
            }
    )
    @Override
    public String first() {
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/");

        System.out.println("本次访问的 service 是：  " + sb.toString());

        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.GET, null, type);
        String result = response.getBody();

        return result;
    }

    /**
     * @Cacheable - 将当期方法的返回值缓存到 cache 中。
     *  只要方法增加了@Cacheable 注解，每次调用当前方法的时候，spring cloud
都会先访问 cache 获取数据，
     *  如果 cache 中没有数据，则访问远程服务获取数据。远程服务返回数据，先保
存在 cache 中，再返回给客户端。
     *  如果 cache 中有数据，则直接返回 cache 中的数据，不会访问远程服务。
```

```
     *  请求缓存会有缓存数据不一致的可能。缓存数据过期、失效、脏数据等情况。
     *  一旦使用了请求缓存来处理幂等性请求操作。则在非幂等性请求操作中必须管理缓存。避免缓存数据的错误。
     * @return
     */
    @Override
    @Cacheable("myCache")
    public String testGet() {
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/testGet");

        System.out.println("本次访问的 service 是:  " + sb.toString());

        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.GET, null, type);
        String result = response.getBody();

        return result;
    }

    /**
     * @CacheEvict - 管理缓存。根据参数 key 删除缓存中对应的缓存数据。
     * @return
     */
    @Override
    @CacheEvict("myCache")
    public String testPost() {
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/testPost");

        System.out.println("本次访问的 service 是:  " + sb.toString());
```

```
        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.POST, null, type);
        String result = response.getBody();

        return result;
    }

    // 本地容错方法，只有远程服务调用过程出现问题的时候，才会调用此方法，获取托底数据。保证服务完整性。
    private String downgradeFallback(){
        return "服务降级方法返回托底数据";
    }
}
```

4 application client 配置文件 application.yml

```
server:
  port: 8082

spring:
  application:
    name: application-client
  redis:
    host: 192.168.89.129

eureka:
  client:
    service-url:
      defaultZone:
        - http://localhost:8761/eureka/

hystrix:
  command:
    default:  #default 全局有效，service id 指定应用有效，如：application-service
      execution:
        timeout:
          # 如果 enabled 设置为 false，则请求超时交给 ribbon 控制，为 true，则超时
```

```
作为容错根据
          enabled: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000 # 超时时间，默认 1000ms
```

5 application client 启动类

```
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;

/**
 * @EnableCircuitBreaker - 开启 Hystrix 容错处理能力。
 *  如果不使用此注解，服务代码中的@HystrixCommand 注解无效。
 * @EnableCaching - 开启 spring cloud 对 cache 的支持。
 * 可以自动的使用请求缓存，访问 redis 等 cache 服务。
 */
@EnableCaching
@SpringBootApplication
@EnableCircuitBreaker
public class ClientApp {
    public static void main(String[] args) {
        SpringApplication.run(ClientApp.class, args);
    }
}
```

五、 Openfeign 的雪崩处理

在声明式远程服务调用 Openfeign 中，实现服务灾难性雪崩效应处理也是通过 Hystrix 实现的。而 feign 启动器 spring-cloud-starter-openfeign 中是包含 Hystrix 相关依赖的。如果只使用服务降级功能则不需要做独立依赖。如果需要使用 Hystrix 其他服务容错能力，需要依赖 spring-cloud-starter-netflix-hystrix 资源。从 Dalston 版本后，feign 默认关闭 Hystrix 支持。所以必须在全局配置文件中开启 feign 技术中的 Hystrix 支持。具体实现如下：

1 服务降级

1.1 POM 依赖

Openfeign 的启动器依赖 spring-cloud-starter-openfeign 中，自带 Hystrix 处理相关依赖，所以不需要修改 POM 依赖，直接通过配置即可开启容错处理机制。

1.2 application client 接口

```
package com.bjsxt.service;

import com.bjsxt.service.impl.FirstClientServiceImpl;
import com.bjsxt.serviceapi.FirstServiceAPI;
import org.springframework.cloud.openfeign.FeignClient;

/**
 * 注解属性 fallback - 当前接口远程调用服务发生问题时，使用哪一个对象中的对应方法用于实现容错逻辑。
 *  Openfeign 技术中，容错处理是使用当前接口的实现类实现的。
 *  实现类中的方法实现，就是对应的容错 fallback 处理逻辑。
 */
@FeignClient(name="openfeign-service", fallback =
FirstClientServiceImpl.class)
public interface FirstClientService extends FirstServiceAPI {
}
```

1.3 application client 接口实现

```
package com.bjsxt.service.impl;

import com.bjsxt.entity.User;
import com.bjsxt.service.FirstClientService;
import org.springframework.stereotype.Component;

import java.util.Arrays;
import java.util.List;

/**
 * 当前类型的实例必须由 Spring 容器管理，需要使用@Component 注解描述。
 * 每个实现方法对应远程服务调用的容错处理逻辑。
 */
```

```java
@Component
public class FirstClientServiceImpl implements FirstClientService {
    @Override
    public List<String> testFeign() {
        return Arrays.asList("Openfeign 返回托底数据");
    }

    @Override
    public User getMultiParams(Integer id, String name) {
        User user = new User();
        user.setId(0);
        user.setUsername("托底数据");
        user.setRemark("getMultiParams");
        return user;
    }

    @Override
    public User postMultiParams(Integer id, String name) {
        User user = new User();
        user.setId(0);
        user.setUsername("托底数据");
        user.setRemark("postMultiParams");
        return user;
    }

    @Override
    public User postObjectParam(User pojo) {
        User user = new User();
        user.setId(0);
        user.setUsername("托底数据");
        user.setRemark("postObjectParam");
        return user;
    }
}
```

1.4 application client 配置文件 application.yml

```yaml
server:
  port: 8082

spring:
  application:
```

```yaml
    name: openfeign-client

eureka:
  client:
    service-url:
      - http://localhost:8761/eureka/
feign: # 开启 feign 技术中的 Hystrix 容错处理机制
  hystrix:
    enabled: true

hystrix:
  command:
    default:
      execution:
        timeout:
          enable: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000
```

1.5 applicaiton client 启动类

```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;

/**
 * @EnableCircuitBreaker - 在 Openfeign 开发的 application client 应用的启动类中，不需要使用此注解开启容错处理逻辑。
 *  Openfeign 的启动器 spring-cloud-starter-openfeign 中，依赖的是 Hystrix 相关 jar 包，不是完整的 spring-cloud-starter-netflix-hystrix 启动器。
 *  如果使用此注解，在启动的时候，会抛出 ClassNotFoundException。
 */
@SpringBootApplication
@EnableFeignClients(basePackages = "com.bjsxt.service")
public class OpenfeignClientApp {
    public static void main(String[] args) {
        SpringApplication.run(OpenfeignClientApp.class, args);
    }
}
```

2 服务熔断

在 Openfeign 技术中，服务熔断对代码和依赖没有其他任何要求，只需要增加对应的配置即可，具体配置如下：

```yaml
server:
  port: 8082

spring:
  application:
    name: openfeign-client

eureka:
  client:
    service-url:
      - http://localhost:8761/eureka/
feign:
  hystrix:
    enabled: true

hystrix:
  command:
    default:
      execution:
        timeout:
          enable: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000
      fallback:
        enabled: true # 当远程调用失败或者请求被拒绝，是否会尝试调用 fallback 方法 。默认 true
      circuitBreaker: # 服务熔断（Circuit Breaker）相关配置属性
        enabled: true # 是否开启熔断器。默认 true
        requestVolumeThreshold: 20 # 默认 20 个;10s 内请求数大于 20 个时就启动熔断器，当请求符合熔断条件时将触发 fallback 逻辑
        errorThresholdPercentage: 50 # 请求错误率大于 50%时就熔断，然后 for 循环发起请求，当请求符合熔断条件时将触发
        sleepWindowInMilliseconds: 5000 # 默认 5 秒;熔断多少秒后去尝试请求
        forceOpen: false # 是否强制打开熔断器， 默认 false
        forceClosed: false # 是否强制关闭熔断器， 默认 false
```

六、 可视化的数据监控 Hystrix-dashboard

Hystrix Dashboard 是一款针对 Hystrix 进行实时监控的工具，通过 Hystrix Dashboard 我们可以在直观地看到各 Hystrix Command 的请求响应时间, 请求成功率等数据。

具体开启方式如下：

1 POM 依赖

Hystrix Dashboard 是针对 Hystrix 容错处理相关数据的监控工具。只要在使用了 Hystrix 技术的应用工程中导入对应依赖即可。注意：如果在 Openfeign 技术中开启 Hystrix Dashboard 监控，则需要将 spring-cloud-starter-netflix-hystrix 启动器导入 POM 文件，否则无法在启动类上使用@EnableCircuitBreaker 注解。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationclient</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
        </dependency>
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-hystrix-dashboard</artifactId>
        </dependency>
    </dependencies>

</project>
```

2 配置文件 application.yml

```yml
server:
  port: 8082

spring:
  application:
    name: application-client
  redis:
    host: 192.168.89.129

eureka:
  client:
    service-url:
      defaultZone:
        - http://localhost:8761/eureka/

hystrix:
  command:
    default:  #default 全局有效,service id 指定应用有效,如:application-service
      execution:
        timeout:
          # 如果 enabled 设置为 false, 则请求超时交给 ribbon 控制,为 true,则超时
作为容错根据
          enabled: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000 # 超时时间, 默认 1000ms
management:
```

```yaml
  endpoints:
    web:
      exposure:
        include: # 开启的actuator监控路径，默认开启info和health。其他需要手工增加，使用*代表开启所有监控路径。
          - info
          - health
          - hystrix.stream
```

3 启动类

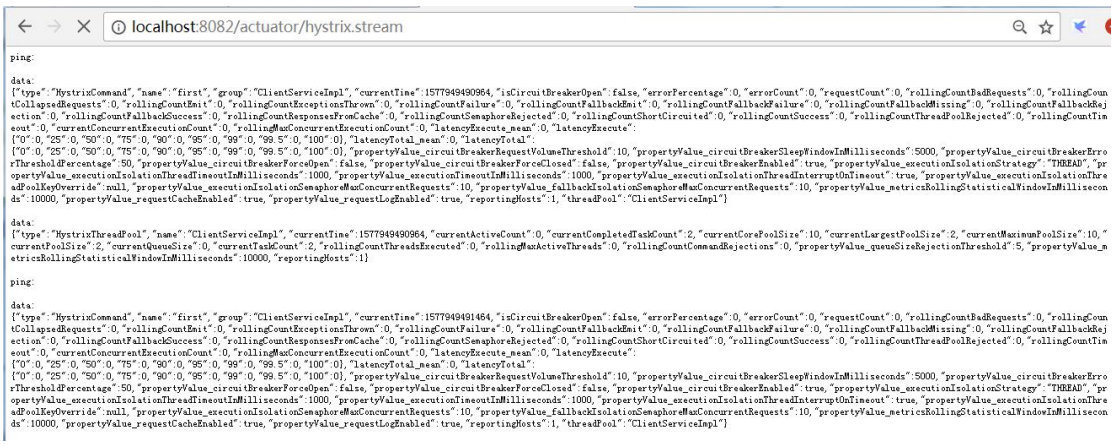
```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
import
org.springframework.cloud.netflix.hystrix.dashboard.EnableHystrixDashboard;

/**
 * @EnableCircuitBreaker - 开启Hystrix容错处理能力。
 *  如果不使用此注解，服务代码中的@HystrixCommand注解无效。
 *  且Hystrix相关监控数据无法收集。
 * @EnableCaching - 开启spring cloud对cache的支持。
 * 可以自动的使用请求缓存，访问redis等cache服务。
 * @EnableHystrixDashboard - 开启Hystrix Dashboard监控仪表盘。
 */
@EnableHystrixDashboard
@EnableCaching
@SpringBootApplication
@EnableCircuitBreaker
public class ClientApp {
    public static void main(String[] args) {
        SpringApplication.run(ClientApp.class, args);
    }
}
```

4 访问 Hystrix 监控数据

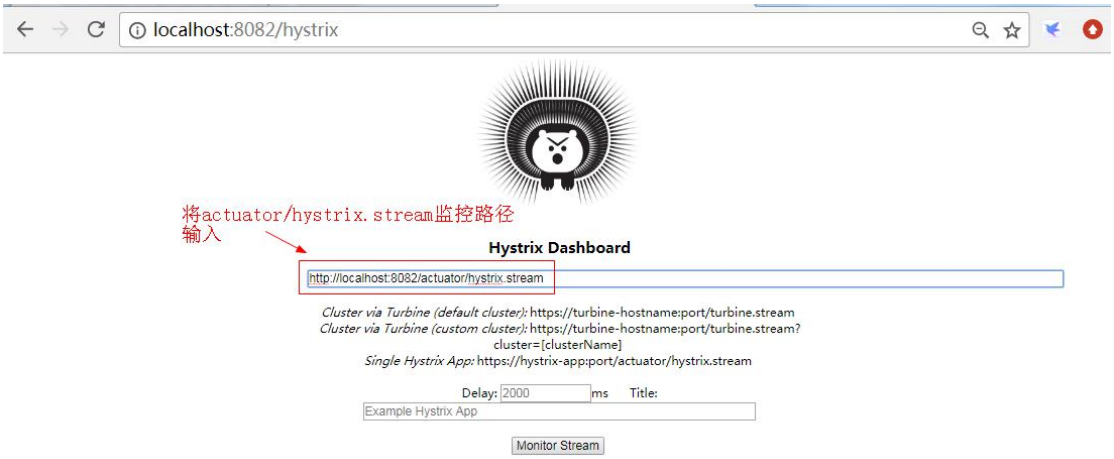
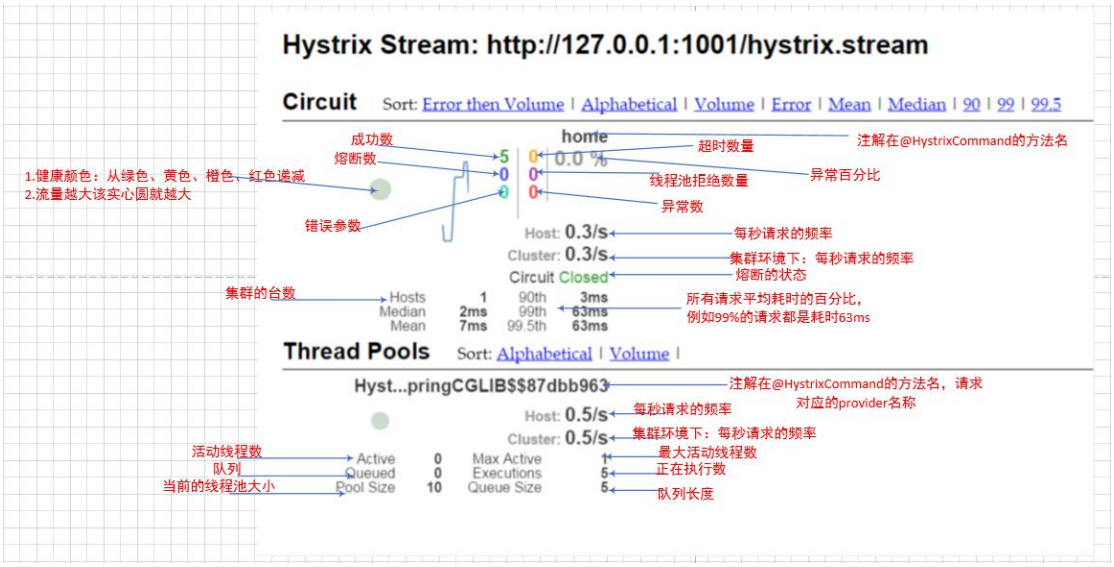
通过浏览器访问提供监控访问路径的应用，具体地址为：

http://ip:port/actuator/hystrix.stream

得到的监控结果如下：



这种监控数据的获取都是 JSON 数据。且数据量级较大。不易于查看。可以使用 Hystrix Dashboard 提供的视图界面来观察监控结果。视图界面访问路径为： http://ip:port/hystrix。



进入后的监控视图界面具体含义如下：